

BotonGrupo1



BotonGrupo1

Revisa y analiza los siguientes códigos, corresponden a una clase en c++ que será utilizada para microcontroladores Raspberry Pi Pico

Menciona, lo bueno, lo malo, que mejorar y que eliminar

El código presenta una base funcional orientada a objetos para manejar botones de forma no bloqueante, pero contiene errores lógicos críticos en el manejo de estados que impedirán su correcto funcionamiento en la práctica.

Lo bueno

Ejecución no bloqueante: Utilizar `to_ms_since_boot(get_absolute_time())` evita usar retardos fijos, liberando al microcontrolador para ejecutar otras tareas concurrentes. CPP

Filtro antirrebote: La ventana de 50 ms implementada en el método `leer()` es una excelente práctica para limpiar el ruido mecánico inherente a los pulsadores físicos. CPP

Inicialización limpia: El constructor configura directamente las direcciones y los pines correspondientes del GPIO utilizando los métodos nativos del SDK de la Raspberry Pi Pico. CPP

Lo malo

Variables locales destructivas: La variable `esperandoDoblePresionado` se declara e inicializa a falso en cada ciclo dentro de `leer()`, lo que impide registrar y recordar un primer clic. CPP

Error de sintaxis: La declaración `bool doblePresionado(tiempoDoblePresionado);` provocará un error de compilación al carecer de tipo y no coincidir con la firma sin parámetros del archivo principal. H + 2

Lógica de switch inutilizada: El método `mantenerPresionado()` evalúa `estadoAnterior`, pero este valor ya fue sobrescrito al final de `leer()`, por lo que las condiciones de cambio jamás se cumplirán. CPP + 1

Qué mejorar

Encapsulamiento de clase: Actualmente, todos los atributos y estados de control están expuestos como `public`. Deberían moverse a la sección `private` para evitar manipulaciones accidentales desde el programa principal. H

Resistencias de hardware: En lugar de exigir resistencias físicas externas en la protoboard, puedes activar las internas del microcontrolador añadiendo `gpio_pull_down(pin)` o `gpio_pull_up(pin)` directamente en el constructor. H + 2

Qué eliminar

Comentarios redundantes: Borra las explicaciones de sintaxis básica de C++ (como `// != significa diferente` o `// la hacemos public`), ya que sobrecargan la lectura sin aportar valor lógico. CPP + 1

Declaraciones locales de estado: Elimina la variable `esperandoDoblePresionado` del interior del método `leer()` y muévela al archivo de cabecera como un

cómo puedo mejorar la logica de void Boton::mantenerPresionado()

El problema principal de la lógica actual es que la condición `else if (presionado && estadoAnterior)` se cumple continuamente mientras el botón se mantiene pulsado. Esto provoca que `estadoMantenerPresionado` se apague milisegundos después de encenderse, parpadeando incontrolablemente en lugar de funcionar como un interruptor fijo (toggle).

Lógica corregida

Para lograr un comportamiento de interruptor real, debes invertir el estado solo una vez por pulsación. Esto se logra separando la acción del tiempo de retención mediante una variable de control.

Añade esta nueva variable a los atributos de tu clase en `BotonGrupo1.h` (puedes dejarla pública para mantener la transparencia en tu arquitectura orientada a objetos):

C++

```
bool clickProcesado = false;
```

Reemplaza el método en `BotonGrupo1.cpp` con la siguiente estructura:

C++

```
void Boton::mantenerPresionado() {
  // Si el botón está presionado y la acción no se ha ejecutado aún
  if (presionado && !clickProcesado) {
    // Invierte el estado actual (de true a false o viceversa)
    estadoMantenerPresionado = !estadoMantenerPresionado;

    // Bloquea futuras ejecuciones mientras el botón siga pulsado
    clickProcesado = true;
  }
  // Al soltar el botón, reinicia el control para el próximo clic
  else if (!presionado) {
    clickProcesado = false;
  }
}
```

Mejoras aplicadas

Independencia de variables frágiles: Elimina el uso de `estadoAnterior`. Al estar programado para sobrescribirse constantemente al final de tu máquina de estados en `leer()`, utilizarlo en funciones externas generaba inconsistencias temporales.

Alternancia limpia: El uso del operador lógico `!` (`!estadoMantenerPresionado`) alterna directamente el valor booleano, ahorrando procesamiento del microcontrolador al eliminar bloques de evaluación innecesarios.

Bloqueo de estado: La bandera `clickProcesado` garantiza que la inversión del switch ocurra en el instante exacto del contacto, ignorando los ciclos de reloj donde tu dedo permanezca sobre el componente.




BotonGrupo1 BotonGrupo1

Revisa nuevamente, para saber que se realizaron bien los cambios

Lamentablemente, los cambios no se aplicaron correctamente y el código aún presenta errores de compilación y fallos lógicos graves que impedirán su funcionamiento.

Aquí están los tres problemas críticos que debes corregir:

1. Variable local destructiva en `mantenerPresionado()`
Declaraste `bool presionadoProcesado = estadoAnterior;` dentro del método. Al ser una variable local, se crea y se destruye cada vez que llamas a la función, perdiendo la memoria de si el clic ya fue procesado o no. CPP

> **Solución:** Debes eliminar esa línea de `BotonGrupo1_2.cpp` y declarar `bool presionadoProcesado = false;` en tu archivo `BotonGrupo1_2.h` junto a las demás variables. CPP + 1

2. Error de sintaxis en `doblePresionado`
Tanto en el archivo `.h` como en el `.cpp`, dejaste `bool doblePresionado(tiempoDoblePresionado);`. Esto causará un error de compilación porque en C++ no puedes pasar una variable como argumento sin definir su tipo, y además, `tiempoDoblePresionado` ya es un atributo de la clase, por lo que no necesitas pasarlo como parámetro. H + 2

> **Solución:** Cámbialo simplemente a `bool doblePresionado();` en ambos archivos.

3. El doble clic sigue roto
Aún mantienes la línea `bool esperandoDoblePresionado = false;` dentro de `void Boton::leer()`. Esto significa que, cientos de veces por segundo, el microcontrolador olvida que estaba esperando un segundo clic. CPP

> **Solución:** Elimina esa línea de `BotonGrupo1_2.cpp` y muévela a los atributos de tu clase en `BotonGrupo1_2.h`, definiéndola como `bool esperandoDoblePresionado = false;`. CPP + 1

Resumen de la estructura correcta para el `.h` **:**

C++

```
// Añade estas variables a BotonGrupo1_2.h
bool presionadoProcesado = false;
bool esperandoDoblePresionado = false;

// Corrige el método a:
bool doblePresionado();
```

Resumen de la estructura correcta para el `.cpp`:

C++

```
// Corrige la firma del método eliminando el parámetro
bool Boton::doblePresionado() {
  if (estadoDoblePresionado) {
    estadoDoblePresionado = false;
    return true;
  }
  return false;
}

// Así debe quedar mantenerPresionado (sin declarar presionadoProcesado ade
void Boton::mantenerPresionado() {
  if (presionado && !presionadoProcesado) {
    estadoMantenerPresionado = !estadoMantenerPresionado;
    presionadoProcesado = true;
  }
  else if (!presionado) {
    presionadoProcesado = false;
  }
}
```

BotonGrupo1 BotonGrupo1

Se encuentra finalmente todo bien?

Casi lo logras, pero aún hay tres errores críticos (dos de compilación y uno de lógica) que impedirán que el código funcione.

Aquí están las correcciones exactas que debes aplicar:

1. Asignación inválida en el archivo `.h`
En `BotonGrupo1_3.h`, intentaste inicializar `presionadoProcesado` usando otra variable de la clase (`estadoAnterior`). Esto no es válido en C++ para inicializaciones por defecto en la definición de la clase. Además, olvidaste inicializar `estadoDoblePresionado`. H+1

Solución en `BotonGrupo1_3.h`**:**
Cambia esas líneas a:

C++

```
bool estadoDoblePresionado = false;
bool esperandoDoblePresionado = false;
bool presionadoProcesado = false;
```

2. Error de sintaxis (punto y coma extra) en el `.cpp`
En `BotonGrupo1_3.cpp`, añadiste un punto y coma después de los paréntesis en la definición de la función. Esto separa la firma del bloque de código, causando un error de compilación inmediato. CPP

Solución en `BotonGrupo1_3.cpp`**:**
Elimina el `;`. Debe quedar así:

C++

```
bool Boton::doblePresionado() {
  // contenido de la función...
```

3. Sobrescritura lógica en `mantenerPresionado()`
Aún mantienes la línea `presionadoProcesado = estadoAnterior;` al principio de la función en `BotonGrupo1_3.cpp`. Esto destruye el propósito de haber hecho `presionadoProcesado` una variable global de la clase, ya que la estás forzando a olvidar su estado en cada ciclo y la igualas a `estadoAnterior` (la cual genera el parpadeo del switch). CPP

Solución en `BotonGrupo1_3.cpp` **:**
Elimina esa línea por completo. La función debe quedar exactamente así:

C++

```
void Boton::mantenerPresionado() {
  if (presionado && !presionadoProcesado) {
    estadoMantenerPresionado = !estadoMantenerPresionado;
    presionadoProcesado = true;
  }
  else if (!presionado) {
    presionadoProcesado = false;
  }
}
```

Una vez apliques estos tres cambios, tu clase compilará y la lógica del botón como interruptor funcionará correctamente.




BotonGrupo1 BotonGrupo1

¡Sí! Finalmente los cambios estructurales y de sintaxis están correctos. Tu código ahora compilará sin errores y la lógica de los estados funcionará como esperas en tu Raspberry Pi Pico.

Has solucionado los problemas críticos: H + 1

Inicialización limpia: Todas las variables de control, incluyendo `estadoDoblePresionado`, `esperandoDoblePresionado` y `presionadoProcesado`, están correctamente declaradas e inicializadas a falso dentro de `BotonGrupo1_4.h`. `H`

Sintaxis corregida: Eliminaste el parámetro indefinido y el punto y coma accidental en la declaración del método `doblePresionado()`, unificando correctamente la firma tanto en el archivo de cabecera como en el de código fuente. `H + 1`

Interruptor (Toggle) funcional: El método `mantenerPresionado()` en `BotonGrupo1_4.cpp` ahora evalúa la variable global `presionadoProcesado` para invertir el estado de `estadoMantenerPresionado` una sola vez por contacto físico. Al soltar el botón, la bandera se reinicia correctamente, lo que elimina el bucle de parpadeo que tenías anteriormente. `CPP + 1`

Tu clase orientada a objetos ya está lista para instanciarse en tu archivo principal y manejar entradas físicas de forma completamente asíncrona.